

# syntax review

A page for reading code, returning to prose, and finding the line you remember. These examples are for visual review. Nothing here needs to run.

Read a few sections in each appearance mode. Toggle source mode with ⌘E to compare the same code inside Markdown fences. Try selecting a few lines, changing a language to plain, and typing a short comment followed by undo.

## typescript: imports, flow, and types

The import, declaration, and control-flow words should be distinguishable without bold type. Comments should remain readable beside the code. Look at the type names, property access, and variables inside template strings.

```
import { readFile } from "node:fs/promises";
import type { Dirent } from "node:fs";

interface Note {
  readonly path: string;
  readonly tags: readonly string[];
  readonly title: string;
}

type Result<T> =
  | { readonly ok: true; readonly value: T }
  | { readonly error: Error; readonly ok: false };

const MAX_PREVIEW_LENGTH = 160;
const headingPattern = /^#\s+(.+)$/m;

// A preview ends on a word so the next visit has a useful
// starting point.
function preview(body: string): string {
```

```

    if (body.length <= MAX_PREVIEW_LENGTH) {
        return body;
    }

    const clipped = body.slice(0, MAX_PREVIEW_LENGTH);
    return `${clipped.slice(0, clipped.lastIndexOf(" "))}...`;
}

export async function readNote(path: string):
Promise<Result<Note>> {
    try {
        const body = await readFile(path, "utf8");
        const match = headingPattern.exec(body);

        if (match === null) {
            throw new Error(`no heading found in ${path}`);
        }

        return {
            ok: true,
            value: { path, tags: ["study", "reading"], title: match[1]
        },
    };
} catch (error) {
    if (!(error instanceof Error)) {
        throw error;
    }

    return { error, ok: false };
}

export function isMarkdown(entry: Dirent): boolean {
    return entry.isFile() && /\.md(?:arkdown)?$/i.test(entry.name);
}

```

The paragraph after a dense block should feel like a return to ordinary reading. Inline `readNote(path)` belongs to the sentence. **A remembered detail** and *a passing thought* should still carry prose emphasis.

## tsx: markup mixed with expressions

Look for a distinction between component names, attributes, and the values inside braces. The text between tags should remain easy to find.

```
import type { ReactNode } from "react";

interface NoteLinkProps {
  children: ReactNode;
  href: string;
  pinned: boolean;
}

export function NoteLink({ children, href, pinned }:
NoteLinkProps) {
  return (
    <article aria-label="saved note" className="note-card">
      <a aria-current={pinned ? "page" : undefined} href={href}>
        {children}
      </a>
      {pinned ? <span className="note-status">pinned</span> :
null}
    </article>
  );
}
```

## rust: lifetimes, patterns, and results

This block puts punctuation next to types and function calls. Check whether references and lifetimes remain visible without competing with the names.

```

use std::collections::BTreeMap;
use std::io;
use std::path::Path;

#[derive(Debug, Clone, PartialEq)]
struct Note<'a> {
    body: &'a str,
    title: &'a str,
}

fn split_note(source: &str) -> Option<Note<'_>> {
    let (first, body) = source.split_once('\n')?;
    let title = first.strip_prefix("# ")?;

    Some(Note { body, title })
}

fn read_titles(paths: &[&Path]) -> io::Result<BTreeMap<String,
String>> {
    let mut titles = BTreeMap::new();

    for path in paths {
        let source = std::fs::read_to_string(path)?;
        match split_note(&source) {
            Some(note) => {
                titles.insert(path.display().to_string(),
note.title.into());
            }
            None => continue,
        }
    }

    Ok(titles)
}

```

# python: a small study index

Follow the comprehension, then the loop below it. The colors should help locate the structure without making a short function feel busy.

```
from collections import defaultdict
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class Note:
    path: Path
    tags: tuple[str, ...]
    title: str

def group_by_tag(notes: list[Note]) -> dict[str, list[str]]:
    grouped: dict[str, list[str]] = defaultdict(list)

    for note in notes:
        for tag in note.tags:
            grouped[tag].append(note.title)

    return {tag: sorted(titles) for tag, titles in
grouped.items()}

def describe(notes: list[Note]) -> str:
    tagged = [note for note in notes if "study" in note.tags]
    count = len(tagged)
    label = "note" if count == 1 else "notes"
    return f"{count} {label} to read again"
```

## sql: reading a query vertically

Find the selected fields, the join, and the filter. Then look away and find the ordering clause again.

```
WITH study_notes AS (  
    SELECT note.path, note.title, note.updated_at  
    FROM note  
    INNER JOIN note_tag ON note_tag.note_path = note.path  
    WHERE note_tag.tag = 'study'  
    AND note.title IS NOT NULL  
)  
SELECT  
    title,  
    path,  
    strftime('%Y-%m-%d', updated_at, 'unixepoch') AS last_read  
FROM study_notes  
ORDER BY updated_at DESC, title ASC  
LIMIT 12;
```

## json and yaml: keys beside values

These are sample records, not app settings. Compare quoted and unquoted keys, booleans, numbers, and strings.

```
{
  "archived": false,
  "collection": "things to read again",
  "notes": [
    { "pages": 12, "score": 0.875, "title": "learning by writing"
  },
    { "pages": 8, "score": null, "title": "finding the remembered
detail" }
  ],
  "tags": ["study", "reading"]
}
```

```
collection: things to read again
defaults: &defaults
  archived: false
  tags: [study, reading]
notes:
- <<: *defaults
  title: learning by writing
  pages: 12
  summary: |
    Write the thought while it is fresh.
    Leave enough context to find it later.
```

## css and html: selectors, properties, and text

Check the custom properties and the `var()` calls. The same block includes numbers, units, an attribute selector, and a media query.

```

.study-note {
  color: var(--foreground);
  background: var(--background);
  font-size: 1rem;
  line-height: 1.65;
  max-width: 42rem;
  padding: 1.5rem;
}

.study-note[data-state="active"] > h2 {
  border-bottom: 1px solid var(--border);
  margin-block: 1.65em 0.5em;
}

@media (prefers-reduced-motion: reduce) {
  .study-note a {
    transition: none;
  }
}

```

```

<article class="study-note" data-state="active">
  <h2>learning by writing</h2>
  <p>A note can hold a question before it holds an answer.</p>
  <a href="#remembered-detail">return to the remembered
detail</a>
  <time datetime="2026-09-08">september 8</time>
</article>

```

## shell: quotes, flags, and substitutions

Read this as a sample, without running it. Look at the quoted path, the variable references, and the command substitution.

```

notes_dir="./sample-notes"
query="remembered detail"

if [ -d "$notes_dir" ]; then
    count=$(rg --files "$notes_dir" -g '*.md' | wc -l)
    printf '%s notes to search\n' "$count"
    rg --line-number --fixed-strings "$query" "$notes_dir"
else
    printf 'sample notes folder is missing\n' >&2
fi

```

## markdown inside a code block

This fenced example contains its own frontmatter and another code fence. Compare it with the surrounding prose, then inspect the whole note in source mode.

```

---
tags: [study]
...
# a thought worth keeping

Read one paragraph, write one question, and keep `the
useful detail`.

> Which part would be hardest to explain from memory?

- [x] read the example
- [ ] write a note in your own words

```ts
const question = "what changed my understanding?";
```

```

## plain and unsupported labels

Both blocks below should stay unhighlighted. The second should retain `notras-example-language` in the language picker and in the file.

```
const thought = "this is plain text";  
if (thought) return thought;
```

```
remember thought("a small detail")  
when found -> read_again
```

## long lines and unusual characters

The long line should scroll inside its code block. Source mode should wrap it. Look at the accented identifiers and the escaped characters as well.

```
const café = "notes from the café";  
const greeting = "こんにちは";  
const escaped = "first line\nsecond line\t\"quoted\"";  
const path = String.raw`notes\study\reading.md`;  
const rememberedDetail = "A note keeps the context around an  
idea: the question that prompted it, the example that made it  
concrete, the part you were unsure about, and the words you would  
search for when you want to find it again several weeks later.";
```

## after a few minutes

Return to the first code block and find `readNote`. Does the color help you locate it? Can you read the comments without slowing down? Does the paragraph after the block feel comfortable? Leave observations below after trying light and dark mode.

- Light mode:
- Dark mode:
- Source mode:
- Anything that draws too much attention: